

DuocUC® INFORMÁTICA Y TELECOMUNICACIONES

2.4.1 Añadiendo seguridad a nuestro Pipeline CI

DOY0101 - Ingeniería DevOps

FORMAMOS PERSONAS PARA UNA SOCIEDAD MEJOR



CONTENIDOS

01 Herramientas de Seguridad en el Desarrollo Seguro (DevSecOps).

02 Introducción a Snyk y SonarCloud.

03 Integrando Snyk en Github Actions.



01

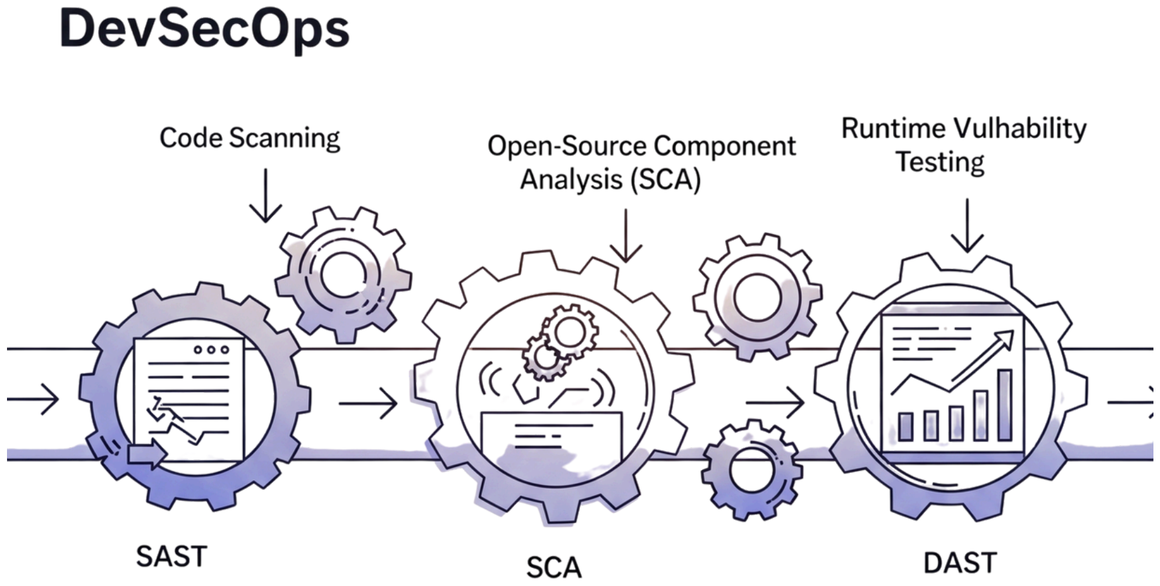
Herramientas de Seguridad en el Desarrollo Seguro (DevSecOps)

Herramientas de Seguridad en el Desarrollo Seguro (DevSecOps)

En el desarrollo seguro bajo el enfoque DevOps, es crucial integrar herramientas de seguridad a lo largo de todo el ciclo de vida del desarrollo de software (SDLC) para detectar vulnerabilidades de manera temprana y garantizar la calidad del código.

Las principales categorías de herramientas son :

- **Software Composition Analysis (SCA)**
- **Static Application Security Testing (SAST)**
- **Dynamic Application Security Testing (DAST)**



SCA vs. SAST vs. DAST: Claves para la Seguridad en DevSecOps

En el desarrollo seguro, integrar análisis de seguridad en el pipeline de CI/CD es fundamental. **SCA, SAST y DAST** son herramientas clave con enfoques distintos que se complementan para ofrecer una cobertura integral:

Característica	SCA (Software Composition Analysis)	SAST (Static Application Security Testing)	DAST (Dynamic Application Security Testing)
Qué Analiza	Componentes de código abierto y bibliotecas de terceros	Código fuente, binario o bytecode (sin ejecutar)	Aplicación en ejecución (simula ataques)
Perspectiva	Caja blanca (dependencias)	Caja blanca (código interno)	Caja negra (funcionalidad externa)
Cuándo se usa	Fases de codificación y construcción (continuo)	Fases tempranas del SDLC (desarrollo, CI)	Fases posteriores (pruebas, pre-producción)
Propósito	Identificar vulnerabilidades en dependencias y cumplir licencias	Detectar vulnerabilidades en el código propio (ej. inyección SQL)	Encontrar vulnerabilidades explotables en tiempo de ejecución (ej. errores de configuración)
Ventajas	Detección temprana en dependencias, gestión de riesgos de terceros	Hallazgos tempranos, cubren el 100% del código propio, reducen costos de corrección	Identifica problemas en el entorno de ejecución, menos falsos positivos, independiente del lenguaje
Ejemplos	Snyk	SonarCloud	Herramientas de pentesting automatizado



02

Introducción a Snyk y SonarCloud

La Seguridad y Calidad en el Desarrollo de Software DevOps

- En la era de la transformación digital, los equipos de desarrollo enfrentan el desafío constante de entregar software confiable, seguro y de alta calidad en ciclos de vida cada vez más cortos.

- En este contexto, **DevOps** se ha consolidado como una metodología clave que promueve la colaboración entre desarrollo y operaciones, permitiendo la integración continua, la entrega automatizada y la supervisión constante del producto.





La Seguridad y Calidad en el Desarrollo de Software DevOps

- Un componente esencial del enfoque DevOps es garantizar la **seguridad** y **calidad** del software desde las primeras etapas del desarrollo.

- Para lograrlo, es indispensable incorporar herramientas y prácticas que permitan detectar vulnerabilidades y evaluar métricas de calidad en tiempo real.

- Herramientas como **SonarCloud**, para análisis de código y cobertura, y **Snyk**, para la identificación de vulnerabilidades en dependencias y contenedores, son aliados estratégicos en este proceso.

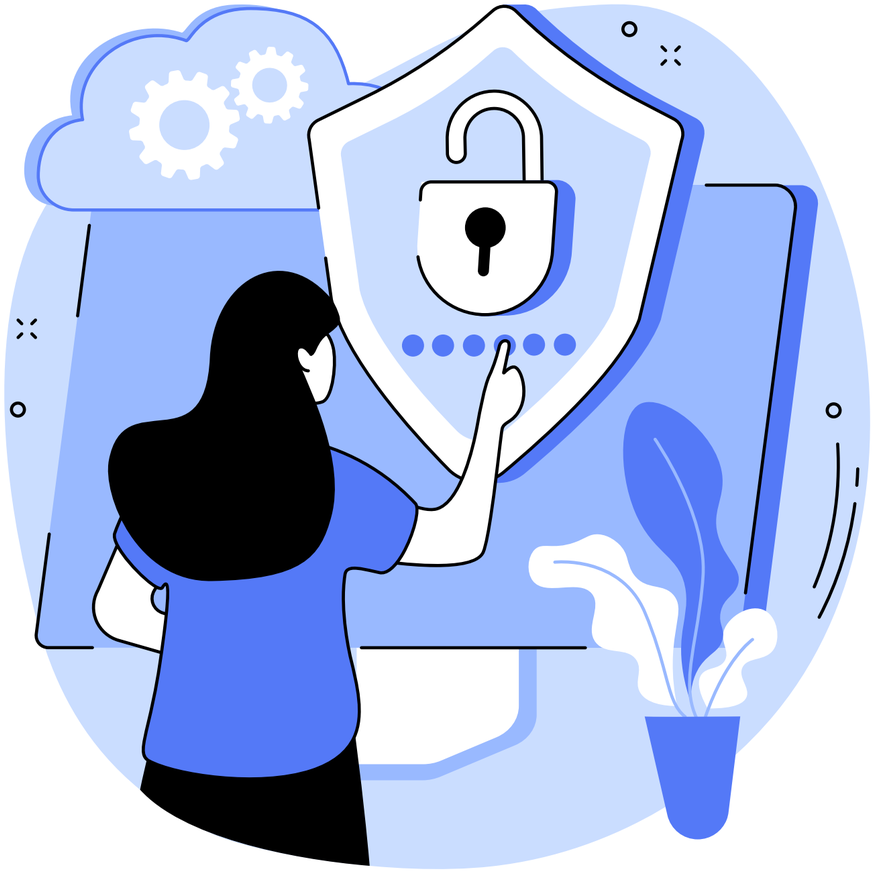
Snyk

- **Snyk** es una poderosa herramienta de análisis de seguridad que permite identificar de manera temprana prácticas inseguras, dependencias obsoletas y vulnerabilidades en bibliotecas utilizadas durante el desarrollo de software.

- Su enfoque va más allá del código estático, ofreciendo también la capacidad de escanear imágenes Docker y contenedores para detectar configuraciones o componentes inseguros.



Snyk



- Diseñada para integrarse fácilmente en flujos de trabajo DevOps, **Snyk ayuda a los desarrolladores a mantener proyectos más seguros y actualizados.** La herramienta no solo detecta problemas, sino que también sugiere soluciones prácticas y rápidas, optimizando el tiempo y esfuerzo invertido en la remediación.

- Para aprovechar sus funcionalidades, es necesario crear una cuenta en su sitio oficial (Snyk.io) y configurarla dentro del pipeline de integración continua. Esta integración asegura que cada cambio en el código o en los contenedores sea evaluado en tiempo real, permitiendo una detección y corrección proactiva de vulnerabilidades y riesgos de seguridad.

- Con Snyk, los equipos de desarrollo pueden construir software más seguro sin comprometer la velocidad ni la eficiencia.

SonarCloud

- **SonarCloud** es una herramienta avanzada de análisis de código que ayuda a los desarrolladores a garantizar la calidad y seguridad de sus aplicaciones. Este servicio basado en la nube permite detectar errores, identificar problemas de calidad, y evaluar métricas clave como la cobertura de pruebas, deuda técnica y adherencia a estándares de codificación.

- Además de ofrecer una visión clara y comprensible del estado del código, SonarCloud se integra fácilmente en flujos de trabajo DevOps, facilitando la supervisión continua de proyectos a lo largo de su ciclo de desarrollo.

sonarcloud

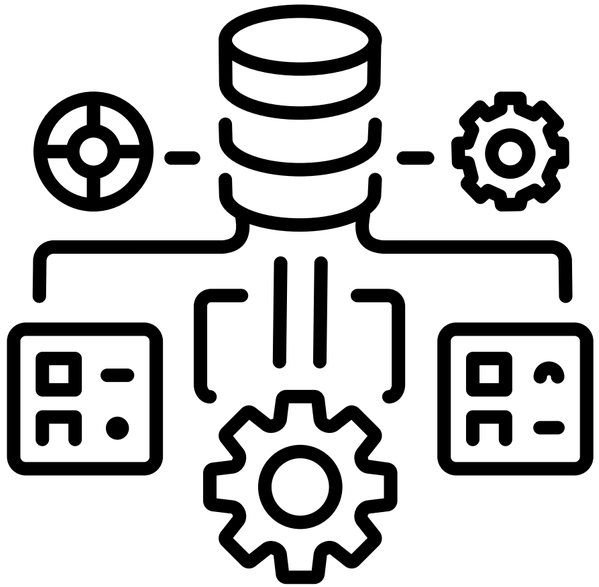


SonarCloud

- Su compatibilidad con herramientas como GitHub Actions y Azure DevOps permite configurar pipelines automatizados que analizan el código con cada cambio, fomentando prácticas de desarrollo más seguras y eficientes.

- Para comenzar a usar SonarCloud, basta con crear una cuenta en su plataforma oficial (SonarCloud.io) y vincular el proyecto deseado, ya sea desde un repositorio público o privado.

- Con su enfoque en la mejora constante y su capacidad para integrarse en entornos colaborativos, SonarCloud se convierte en un recurso esencial para garantizar la calidad en el desarrollo moderno de software





03

Integrando Snyk en Github Actions

Integrando Snyk en Github Actions

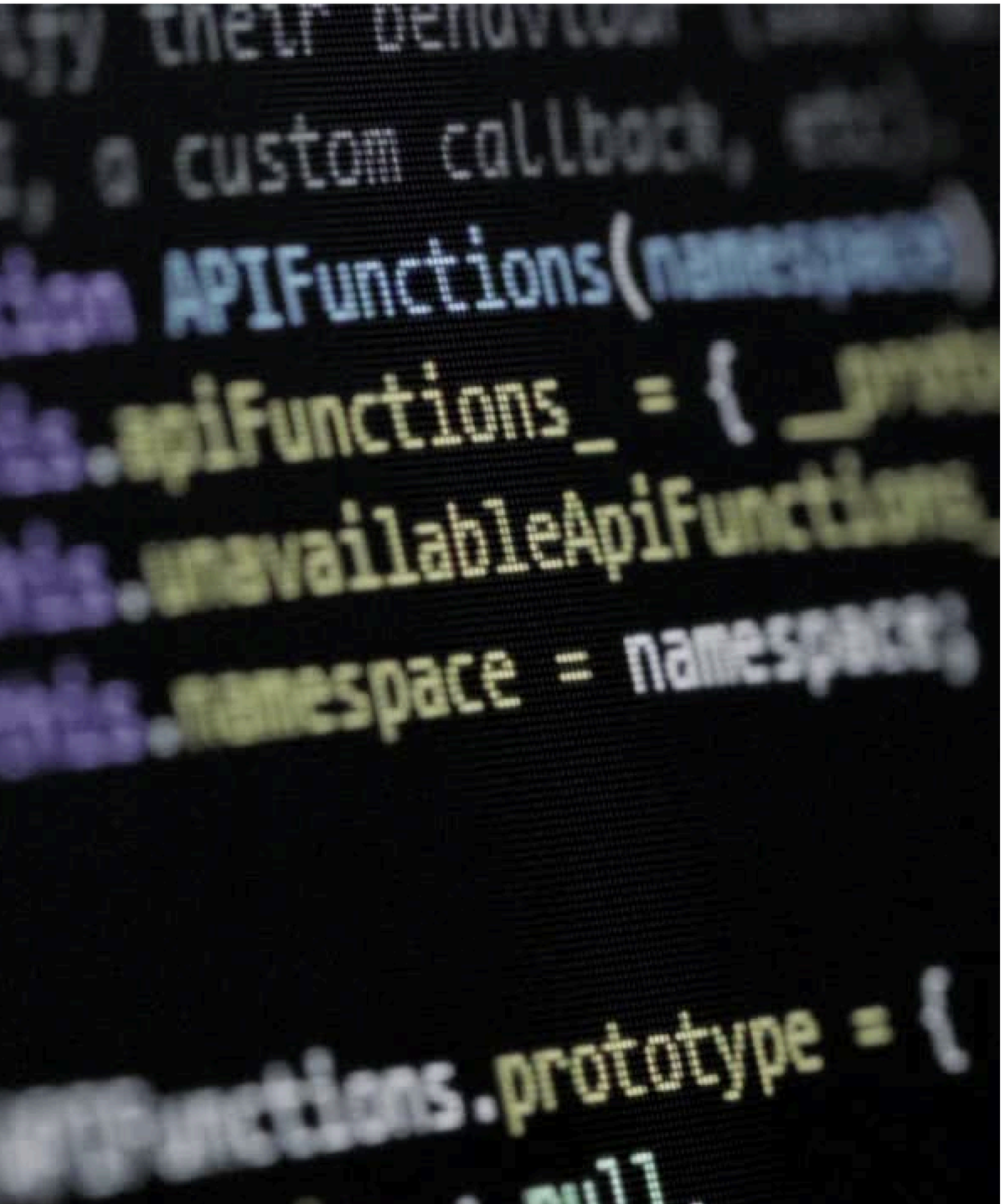
- Como ya hemos visto en nuestro pipeline de github actions podemos separar nuestras ejecuciones por tareas (jobs), donde cada tarea tiene responsabilidades distintas.

- En este caso integraremos una tarea para que Snyk pueda analizar nuestro proyecto y podamos obtener un análisis del estado del mismo. Y de acuerdo con esto, poder definir si nuestra ejecución continúa o no.





GitHub Actions

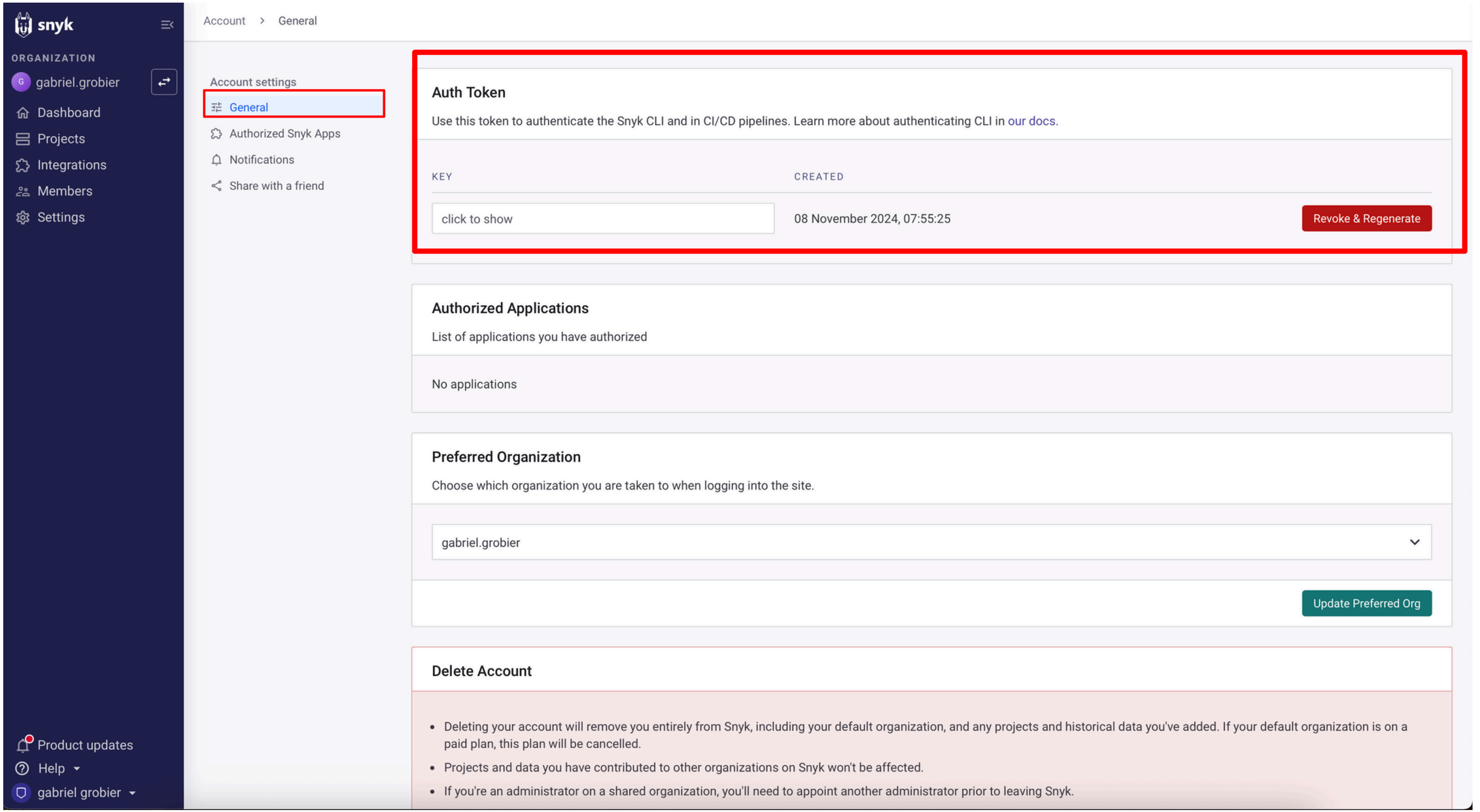


Integrando Snyk en Github Actions

- Para poder llevar esto a cabo primero debemos iniciar sesión en nuestra cuenta de Snyk y luego debemos ir a la configuración de la cuenta que podemos ingresar directo desde su enlace https://app.snyk.io/account

- En la configuración podremos generar nuestro Token de cuenta que nos permitirá dar permisos de uso a nuestro pipeline, es importante mantener seguro este token ya que, con él, cualquier persona podría utilizar funcionalidades de snyk como si fuésemos nosotros

Integrando Snyk en Github Actions



Integrando Snyk en Github Actions

Cuando obtenemos el Token desde nuestra cuenta podemos ir a Github Actions y configurarlo como secreto para poder empezar a crear nuestro pipeline de actions como el siguiente ejemplo

```yaml
- name: Authenticate Snyk CLI
  env:
    SNYK_TOKEN: ${{ secrets.SNYK_TOKEN }}  # Token guardado en GitHub Secrets
  run: snyk auth $SNYK_TOKEN

- name: Snyk test Java dependencies
  run: snyk test --severity-threshold=high  # Analiza vulnerabilidades de severidad alta o mayor

- name: Generate Snyk JSON Report
  if: always()
  run: |
    echo "Generando reporte JSON de Snyk..."
    snyk test --json > snyk-report.json || echo "Falló snyk test, pero se continúa"
    ls -lh snyk-report.json || echo "No se generó el reporte"

- name: Upload Snyk JSON Report
  if: always()
  uses: actions/upload-artifact@v4
  with:
    name: snyk-report
    path: snyk-report.json
```

Integrando Sonarcloud en Github Actions





- Como ya hemos visto, en nuestro pipeline de GitHub Actions podemos separar nuestras ejecuciones por tareas (*jobs*), donde cada tarea cumple funciones específicas.

- En este caso, integraremos una tarea dedicada a **SonarCloud**, que nos permitirá realizar un análisis de calidad del código de nuestro proyecto. Con base en los resultados, podremos decidir si el flujo de integración continua continúa o se detiene.

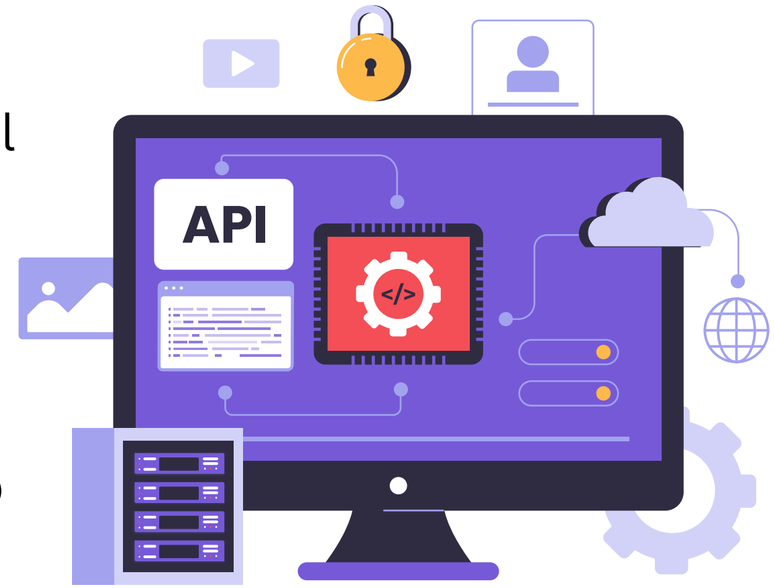
Integrando Sonarcloud en Github Actions

- Para llevar esto a cabo, primero debemos tener una cuenta en SonarCloud. Una vez dentro, debemos vincular nuestro repositorio (por ejemplo, de GitHub) a SonarCloud y luego generar un **token de autenticación** desde la sección de seguridad de nuestra cuenta: https://sonarcloud.io/account/security/

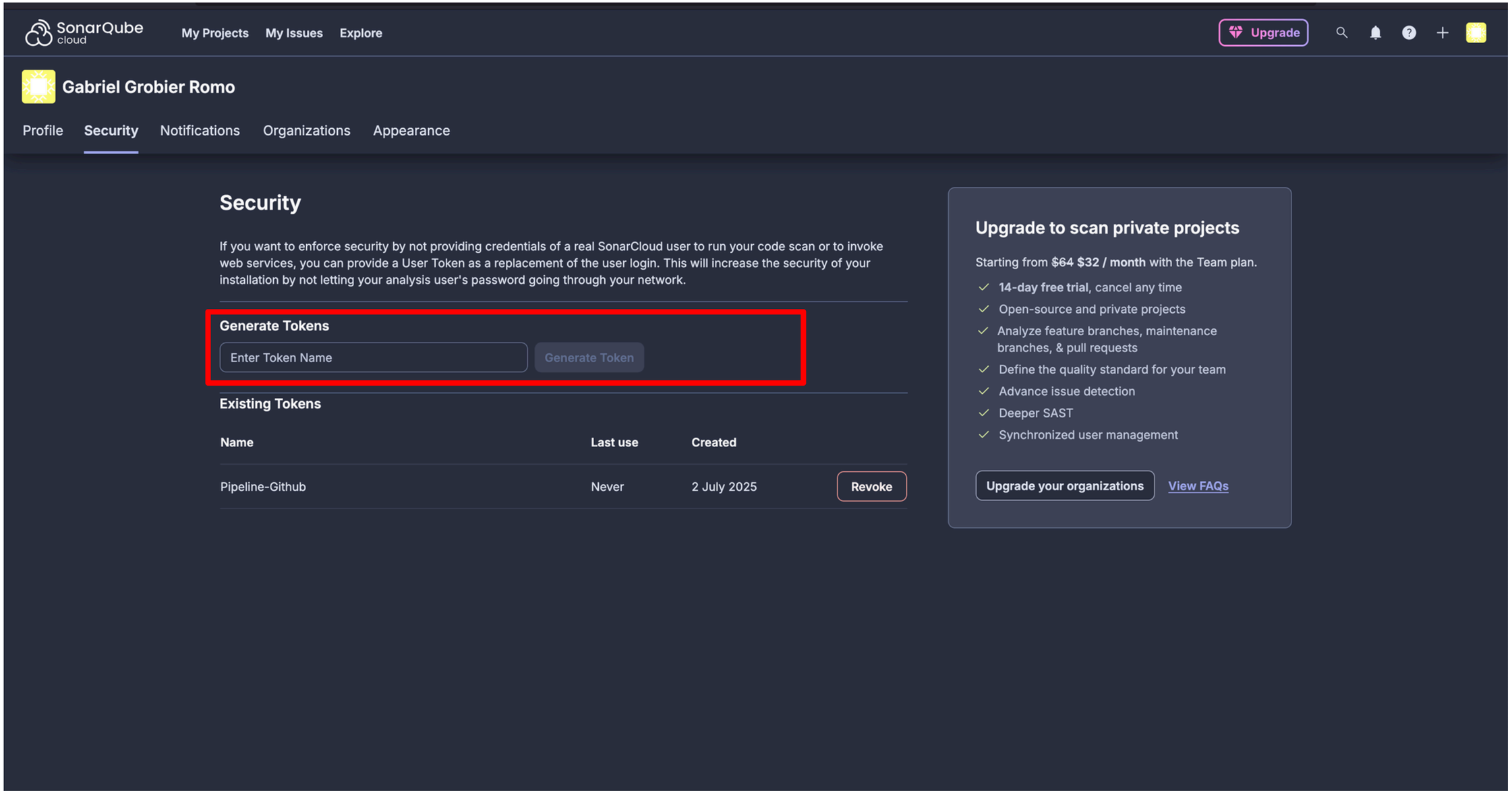
- Este token será utilizado por nuestro pipeline para autenticar el análisis. Es importante mantener este token seguro, ya que otorga acceso a las funcionalidades de SonarCloud con permisos de nuestra cuenta.

- Una vez generado el token, deberemos almacenarlo como un **secreto** en nuestro repositorio de GitHub (por ejemplo, bajo el nombre SONAR_TOKEN), y luego configurar el archivo del workflow para incluir el análisis con el plugin de SonarCloud para Maven o Gradle, según corresponda a nuestro proyecto.

sonarcloud



Integrando Sonarcloud en Github Actions



Integrando Sonarcloud en Github Actions

Luego de tener el token de nuestra cuenta podremos configurar nuestro pipeline.

```yaml

- name: Run SonarCloud Analysis
  env:
    SONAR_TOKEN: ${{ secrets.SONAR_TOKEN }}  # Token SonarCloud desde Secrets
  run: |
    mvn verify sonar:sonar \
      -Dsonar.projectKey=GabrielGrobier_Jacoco-sonacloud \
      -Dsonar.organization=gabrielgrobier \
      -Dsonar.host.url=https://sonarcloud.io \
      -Dsonar.login=${SONAR_TOKEN}

```

Integrando Sonarcloud y Snyk en Github Actions

Llevemos a la práctica e integremos todo dentro del proyecto

PARTE 1

```yaml
name: Spring Boot CI/CD Pipeline

on:
  push:
    branches:
      - main  # El pipeline se ejecuta cuando se hace push a la rama 'main'

jobs:

  # 1. Análisis de Seguridad con Snyk
  snyk-scan:
    name: Snyk Security Scan
    runs-on: ubuntu-latest

    steps:
      - name: Checkout code
        uses: actions/checkout@v4  # Clona el repositorio

      - name: Install Snyk CLI
        run: npm install -g snyk  # Instala la CLI de Snyk

      - name: Authenticate Snyk CLI
        env:
          SNYK_TOKEN: ${{ secrets.SNYK_TOKEN }}  # Token guardado en GitHub Secrets
        run: snyk auth $SNYK_TOKEN

      - name: Snyk test Java dependencies
        run: snyk test --severity-threshold=high  # Analiza vulnerabilidades de severidad alta o mayor

      - name: Generate Snyk JSON Report
        if: always()
        run: |
          echo "Generando reporte JSON de Snyk..."
          snyk test --json > snyk-report.json || echo "Falló snyk test, pero se continúa"
          ls -lh snyk-report.json || echo "No se generó el reporte"

      - name: Upload Snyk JSON Report
        if: always()
        uses: actions/upload-artifact@v4
        with:
          name: snyk-report
          path: snyk-report.json

```

Integrando Sonarcloud y Snyk en Github Actions

Llevemos a la práctica e integremos todo dentro del proyecto

PARTE 2

```yaml
# 2. Pruebas unitarias y cobertura con JaCoCo
  test:
    name: Run Tests with JaCoCo
    runs-on: ubuntu-latest
    needs: snyk-scan  # Solo se ejecuta si pasó el escaneo de Snyk

    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Set up JDK 17
        uses: actions/setup-java@v3
        with:
          distribution: 'temurin'
          java-version: '17'

      - name: Cache Maven dependencies
        uses: actions/cache@v3
        with:
          path: ~/.m2
          key: ${{ runner.os }}-m2-${{ hashFiles('**/pom.xml') }}
          restore-keys: |
            ${{ runner.os }}-m2

      - name: Build and test
        run: mvn clean verify  # Ejecuta los tests y genera reporte de cobertura JaCoCo

      - name: Upload JaCoCo Report
        uses: actions/upload-artifact@v4
        with:
          name: jacoco-report
          path: target/site/jacoco/

```

Integrando Sonarcloud y Snyk en Github Actions

Llevemos a la práctica e integremos todo dentro del proyecto

PARTE 3

```yaml
  sonar-analysis:
    name: SonarCloud Analysis
    runs-on: ubuntu-latest
    needs: test  # Solo se ejecuta si pasaron los tests

    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Set up JDK 17
        uses: actions/setup-java@v3
        with:
          distribution: 'temurin'
          java-version: '17'

      - name: Cache Maven dependencies
        uses: actions/cache@v3
        with:
          path: ~/.m2
          key: ${{ runner.os }}-m2-${{ hashFiles('**/pom.xml') }}
          restore-keys: |
            ${{ runner.os }}-m2


      - name: Run SonarCloud Analysis
        env:
          SONAR_TOKEN: ${{ secrets.SONAR_TOKEN }}  # Token SonarCloud desde Secrets
        run: |
          mvn verify sonar:sonar \
            -Dsonar.projectKey=nombreOrganizacion_NombreProyecto \
            -Dsonar.organization=nombreOrganizacion \
            -Dsonar.host.url=https://sonarcloud.io \
            -Dsonar.login=${SONAR_TOKEN}
```

Integrando Sonarcloud y Snyk en Github Actions

Llevemos a la práctica e integremos todo dentro del proyecto

PARTE 4

```yaml
# 4. Empaquetado final de la aplicación
  build:
    name: Build Jar
    runs-on: ubuntu-latest
    needs: sonar-analysis  # Solo se ejecuta si pasa el análisis de calidad

    steps:
      - name: Checkout code
        uses: actions/checkout@v4

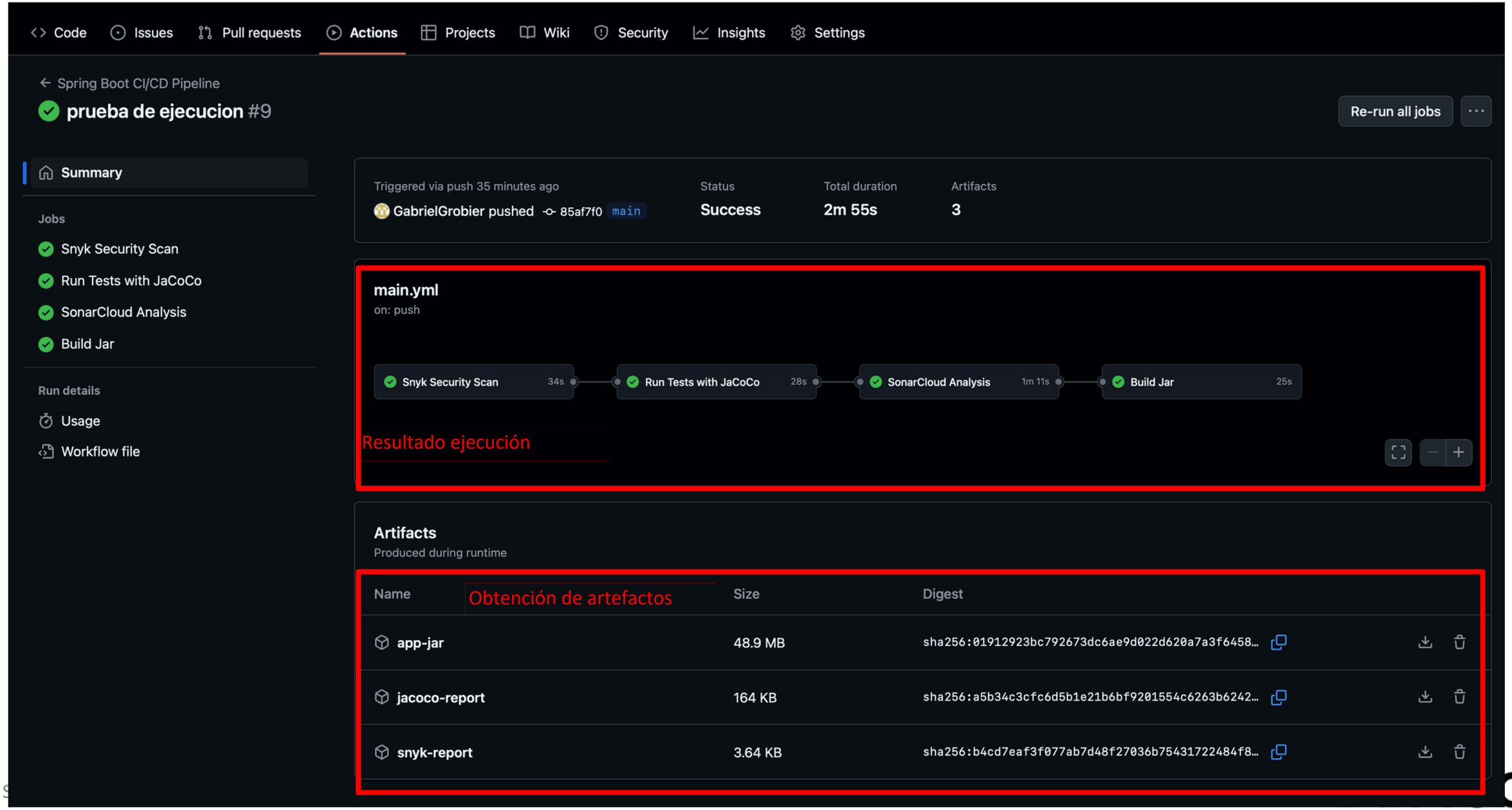
      - name: Set up JDK 17
        uses: actions/setup-java@v3
        with:
          distribution: 'temurin'
          java-version: '17'

      - name: Package application
        run: mvn clean package -DskipTests  # Empaqueta sin volver a ejecutar pruebas

      - name: Upload JAR artifact
        uses: actions/upload-artifact@v4
        with:
          name: app-jar
          path: target/*.jar  # Sube el .jar como artefacto del pipeline

```

Resultado de Ejecución



DuocUC®

CERCANÍA. LIDERAZGO. FUTURO.

7 AÑOS ACREDITADO

Comisión Nacional de Acreditación CNA-Chile

NIVEL DE EXCELENCIA

HASTA OCTUBRE 2031

Docencia de pregrado / Gestión institucional / Aseguramiento interno de la calidad / Vinculación con el Medio / Investigación, creación y/o innovación